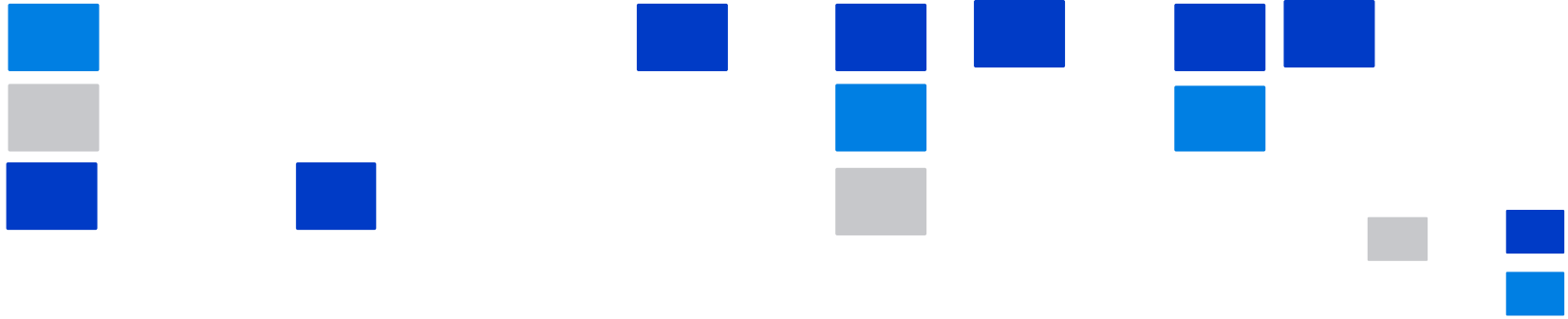




Sistemes Operatius

Tema 4: Crides al sistema - Cues de missatges • **Curs 2026-27**

Comunicació entre processos

Per intercanviar informació entre processos, ho podem fer amb quatre mètodes:

1. Memòria compartida
2. Pas de missatges
3. Pipes
4. Sockets

Memòria compartida:

- Compartició de variables.
- La responsabilitat de comunicació cau en l'usuari.
- El sistema operatiu ofereix la memòria compartida.

Pas de missatges:

- La responsabilitat de comunicació cau en el sistema operatiu.
- La sincronització està implícita.

Avantatges i inconvenients

- En la **memòria compartida**:
 - No hi ha protecció sobre les dades.
 - No hi ha sincronització.
 - Accés directe a les dades.
 - Facilitat de comprensió.
- En el **pas de missatges**:
 - La transferència de dades (`send()`/`receive()`).
 - La sincronització és inherent.
 - S'utilitzen espais de memòria privats.
 - És més segur.
 - Més lent (fa còpies de les dades).

Pas de missatges

- El sistema operatiu ens ofereix unes primitives bàsiques:
 - `send()`
 - `receive()`
- Opcions d'implementació:
 - A) Enllaç directe o indirecte.
 - B) Buffering limitat/il·limitat o sense buffering.
 - C) Mida fixa o variable.

Pas de missatges

A) Enllaç directe o indirecte

A1. Enllaç directe: cada procés que vulgui enviar o rebre haurà de mencionar explícitament l'emissor o receptor d'aquest. Les funcions que tenim són:

```
send(B, msg)
```

```
receive(A, msg)
```

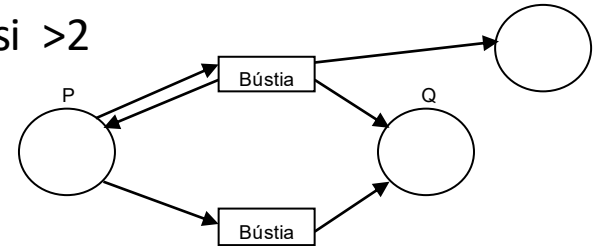
- S'estableix un enllaç automàticament, en cada parell de processos que es comuniquin.
- Un enllaç està associat a 2 processos. Només poden establir un enllaç entre **A** i **B**.
- Els enllaços són bidireccionals.
- Si **A** canvia el seu nom, tots els que rebin d'ell han de canviar el nom amb què reben el missatge.

Pas de missatges

A) Enllaç directe o indirecte

A2. Enllaç indirecte:

- Els missatges s'envien i reben mitjançant bústies (*mailboxes*) o de ports. Una bústia és un objecte on els processos poden dipositar missatges o llegir-los.
- Els missatges s'envien sempre a una bústia, no al procés (la bústia pot pertànyer a un procés o al SO).
- Cada bústia té un "id" que és compartit pels processos.
- S'estableix un enllaç entre 2 o més processos (si >2 processos comparteixen una bústia).



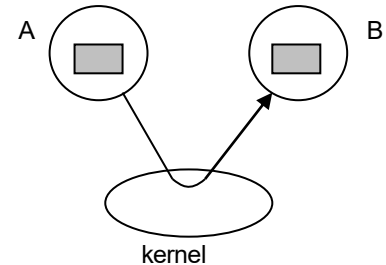
Pas de missatges

B) Buffering limitat/il·limitat o sense buffering

El Buffering és el nombre de missatges que es poden emmagatzemar temporalment en una bústia. Si hi ha buffering entremig dels processos, ens podem trobar amb les següents situacions:

B1. Capacitat zero:

- L'emissor espera al receptor; emissió i recepció del missatge són síncrones.
- En enviar un missatge hi ha una sincronització (*rendez_vous*)
- 1 única còpia del missatge.
- Es bloqueja un procés quan no el rep ningú, o quan està esperant rebre i ningú l'envia.

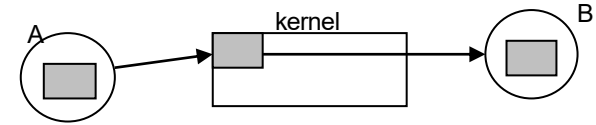


Pas de missatges

B) Buffering limitat/il·limitat o sense buffering

B2. Capacitat limitada (MAX missatges)

- L'emissor només es bloqueja quan la cua està plena. Al fer `send()` no es bloqueja mentre el nombre de missatges sigui $< \text{MAX}$.
- Es fan 2 còpies del missatge. Una de l'origen al *kernel*, i l'altre del *kernel* al destí.
- Es fa una còpia en buffers (estructures) interns.



B3. Capacitat il·limitada

- L'emissor fa *sends* i no es bloqueja mai.
- Es fan 2 còpies del missatge.

Pas de missatges

C) Mida fixa o variable

C1. Mida fixa

- Simplicitat d'implementació.
- Dificulta el treball del programador.
- És el més usual.

C2. Mida variable

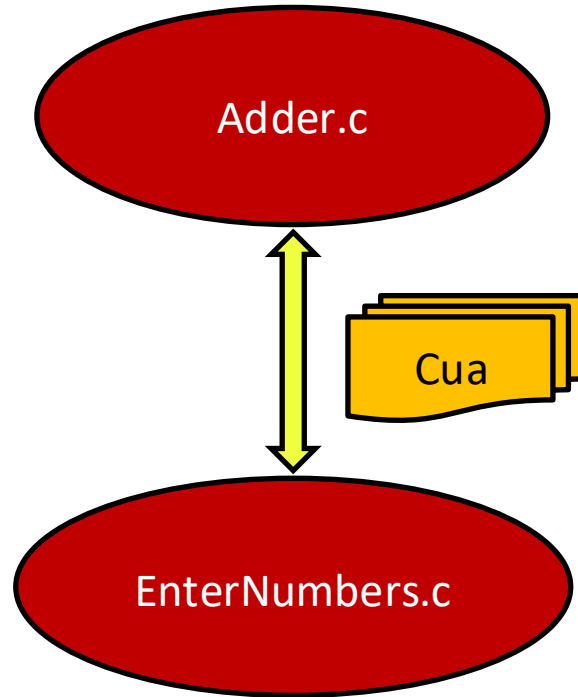
- D'implementació més complexa.
- Facilita el treball del programador.
- Problema: requereix gestió memòria.

Pas de missatges en C

Procediment de funcionament:

1. Definir l'**estructura** dels missatges que es volen enviar i/o rebre. Ha de tenir obligatòriament com **primer camp** un **long** com identificador del missatge.
2. S'obté una **clau** cridant **ftok()** a partir d'un fitxer qualsevol i un enter. Tots els processos que vulguin compartir aquesta cua de missatges, hauran d'usar el mateix fitxer i el mateix enter.
3. Es **crea la cua de missatges** i se n'obté un identificador cridant **msgget()**.
4. **S'envia un missatge** a la cua cridant a **msgsnd()**.
5. **Es rep un missatge** de la cua cridant a **msgrcv()**.
6. **S'esborra i es tanca la cua de missatges** cridant a **msgctl()**.

Exemple en C



EnterNumbers.c

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <unistd.h>
```

```
typedef struct {
    long id_msg;
    int n1, n2;
} Message1;
```

```
typedef struct {
    long id_msg;
    int result;
} Message2;
```

EnterNumbers.c

```
void main() {
    key_t key;
    int id_queue, n;
    Message1 M1;
    Message2 M2;
    char buf[64];

    key = ftok("EnterNumbers.c", 12);
    if (key == (key_t)-1) {
        write(STDERR_FILENO, "Error: Key\n\n", 12);
        exit(-1);
    }

    id_queue = msgget(key, 0600 | IPC_CREAT);
    if (id_queue == -1) {
        write(STDERR_FILENO, "Error creating queue\n\n", 22);
        exit(-1);
    }
}
```

EnterNumbers.c

```
M1.id_msg = 1;
M1.n1 = 1;
while (M1.n1 != 0) {
    write(STDOUT_FILENO, "Enter num1:", 11);
    n = read(STDIN_FILENO, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    M1.n1 = (int)strtol(buf, NULL, 10);
    write(STDOUT_FILENO, "Enter num2:", 11);
    n = read(STDIN_FILENO, buf, sizeof(buf) - 1);
    buf[n] = '\0';
    M1.n2 = (int)strtol(buf, NULL, 10);

    msgsnd(id_queue, (struct msgbuf *)&M1, sizeof(int) * 2, IPC_NOWAIT);
    write(STDOUT_FILENO, "Sending petition...\n", 20);
    if (M1.n1 != 0) {
        msgrcv(id_queue, (struct msgbuf *)&M2, sizeof(int), 2, 0);
        n = sprintf(buf, "%d+%d = %d\n\n", M1.n1, M1.n2, M2.result);
        write(STDOUT_FILENO, buf, n);
    }
}
write(STDOUT_FILENO, "Program finished\n\n", 18);
msgctl(id_queue, IPC_RMID, (struct msqid_ds *)NULL);
```

Adder.c

// La primera part del codi és igual a l'anterior

```
M2.id_msg = 2;
```

```
for (;;) {
```

```
    msgrcv(id_queue, (struct msgbuf *)&M1, sizeof(int) * 2, 1, 0);
```

```
    n = sprintf(buf, "Received: %d,%d\n", M1.n1, M1.n2);
```

```
    write(STDOUT_FILENO, buf, n);
```

```
    if (M1.n1 != 0) {
```

```
        M2.result = M1.n1 + M1.n2;
```

```
        msgsnd(id_queue, (struct msgbuf *)&M2, sizeof(int), IPC_NOWAIT);
```

```
        n = sprintf(buf, "Sent = %d\n", M2.result);
```

```
        write(STDOUT_FILENO, buf, n);
```

```
    } else {
```

```
        exit(1);
```

```
    }
```

```
}
```

```
}
```